7

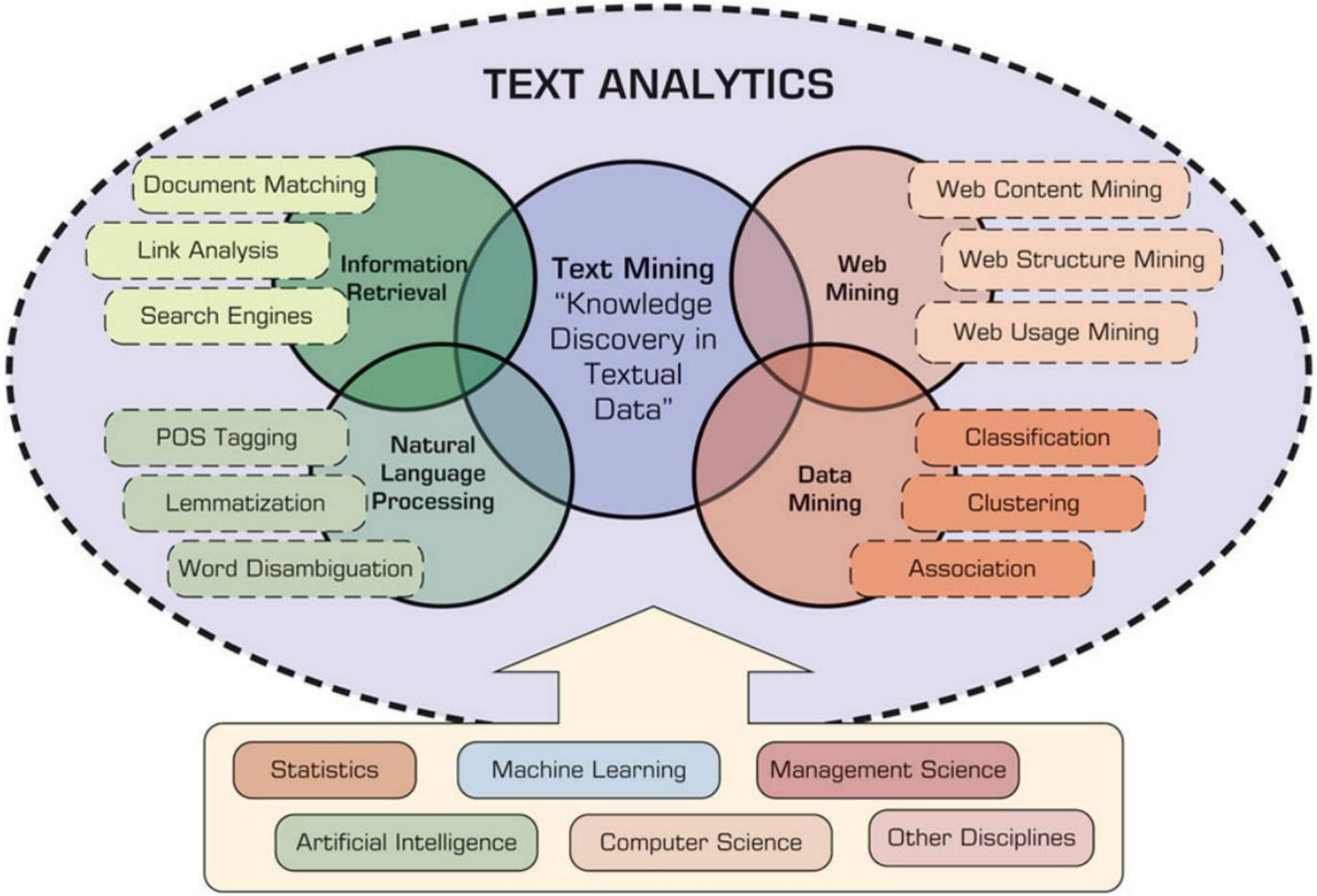
Text Data Cleaning

April, 2026

Rachid BENOUINI

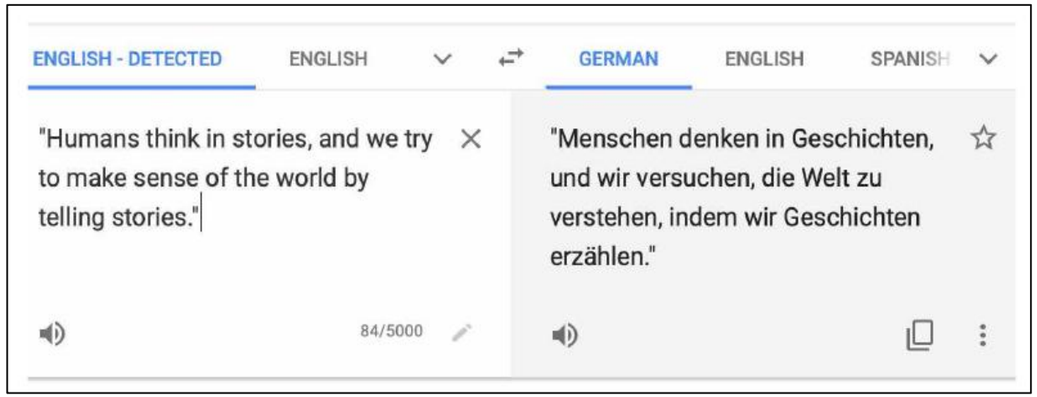
Too Many Terms

- Text Mining
- Text Analytics
- Text Processing
- Information Retrieval
- Information Extraction
- Natural Language Processing
- Computational Linguistics
- Unstructured Data Mining
- …



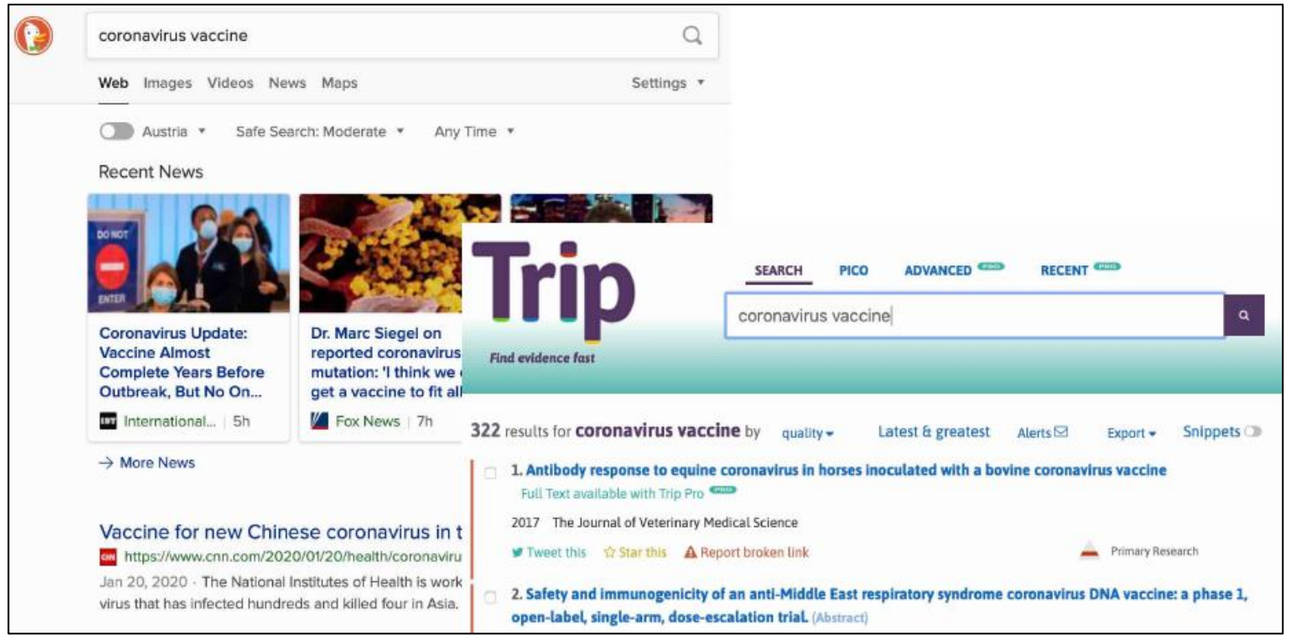
Applications

Machine Translation



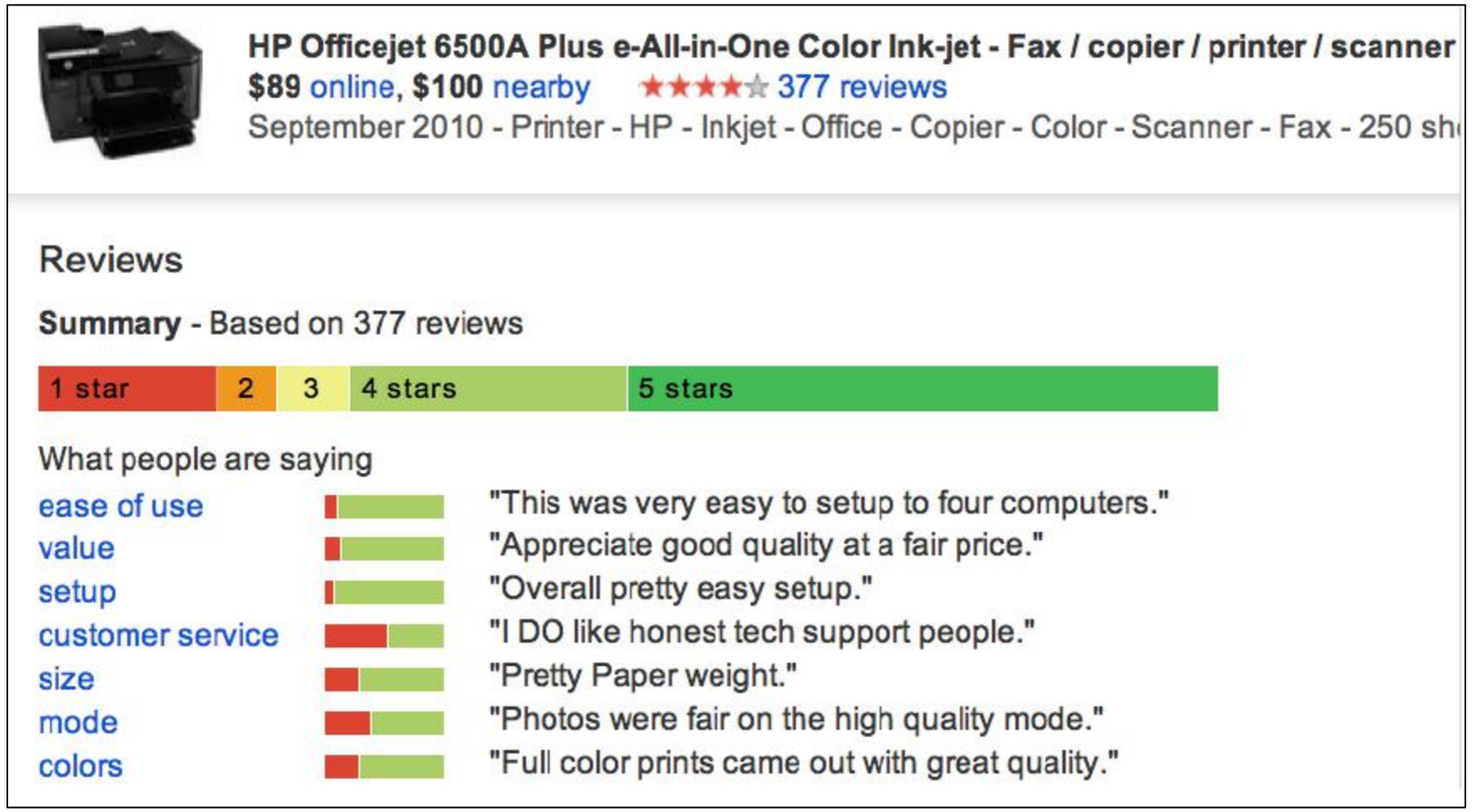
Q&A

Information Retrieval

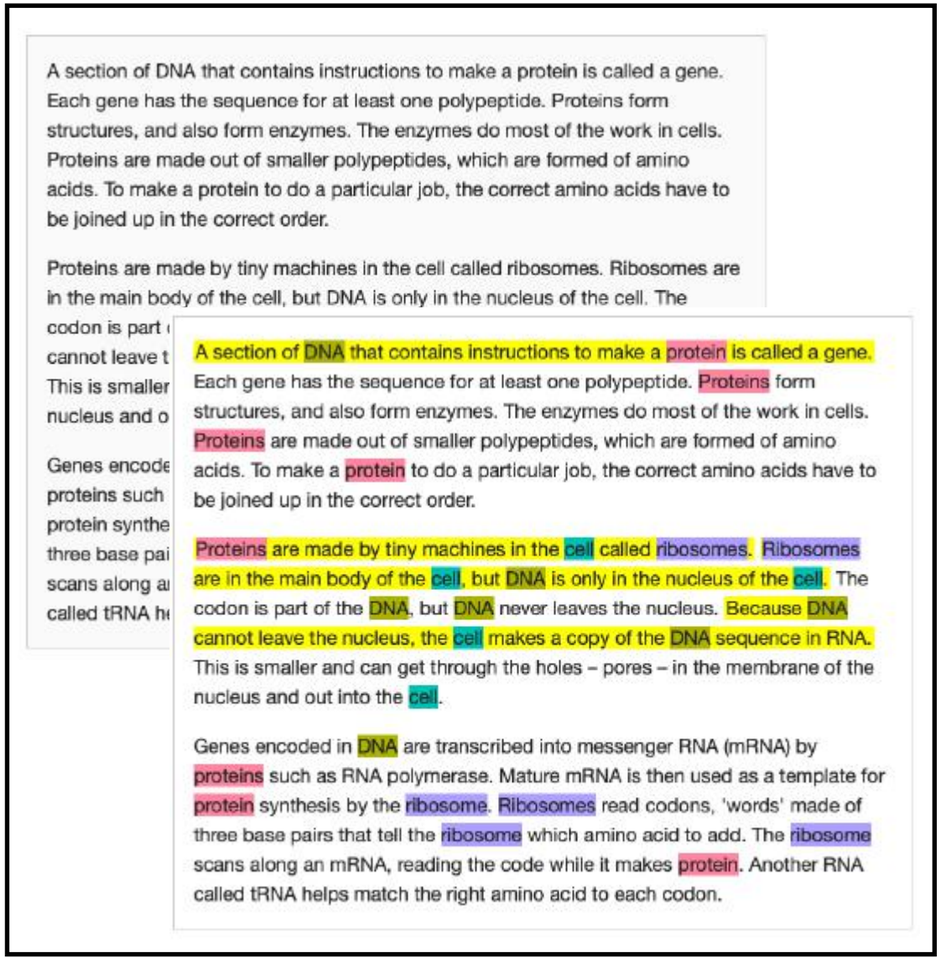


Applications

Market intelligence

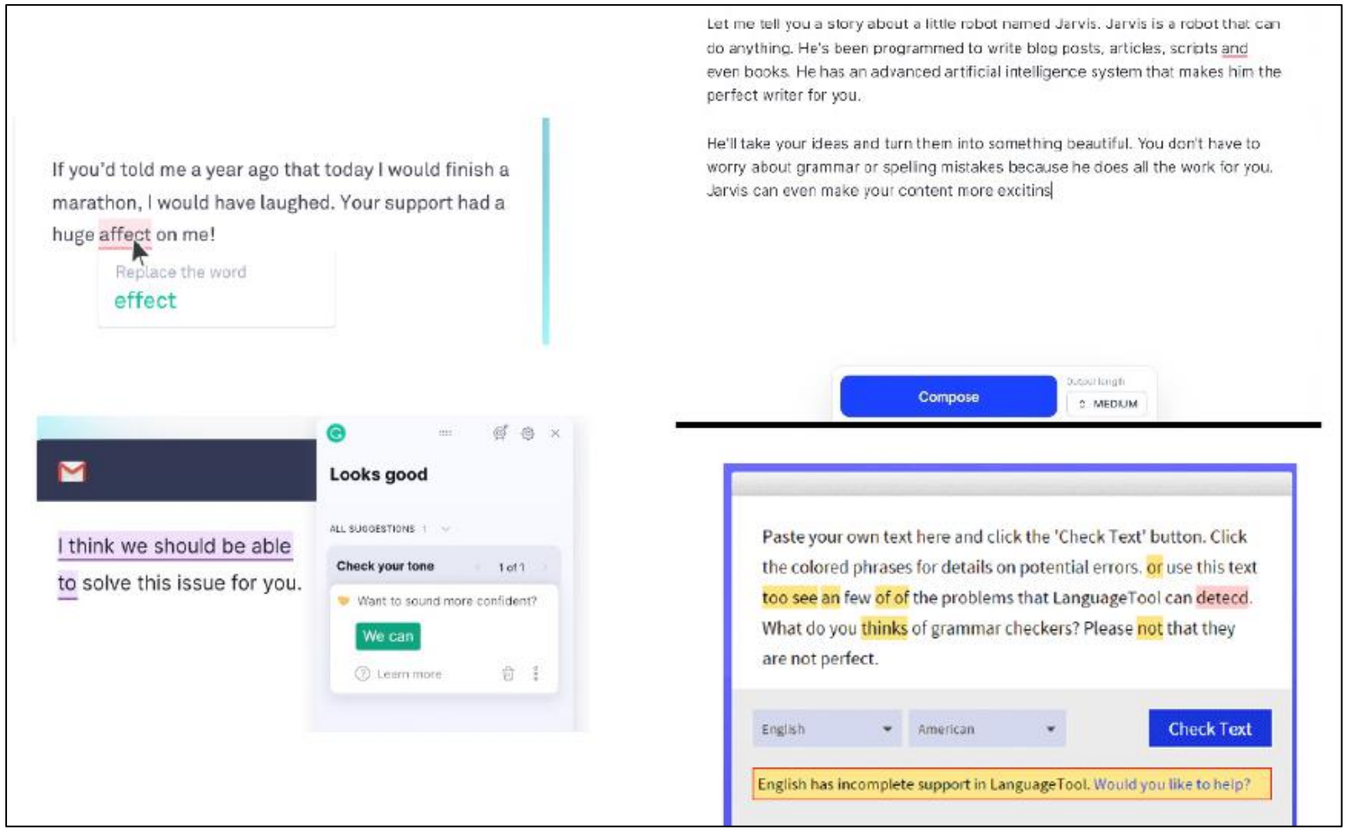


Summarization

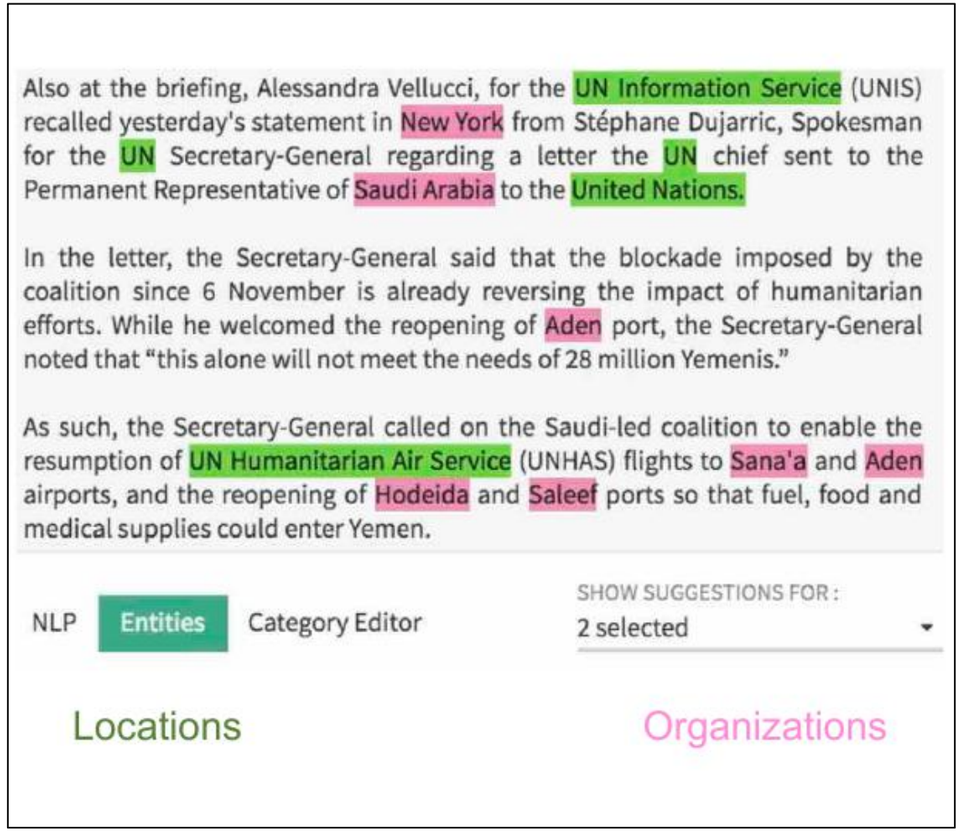


Applications

Content generator / Writing assistant



Named Entity Recognition



What is Text Cleaning ?

- Text Cleaning (or preprocessing) transforms raw, unstructured text into a clean, standardized format suitable for analysis by algorithms.
- It is the first and one of the most crucial steps in any NLP pipeline

Why is it important?

- **Removes noise** and irrelevant information (HTML tags, special characters, formatting artifacts)
- **Standardizes text** to reduce vocabulary size and linguistic variation
- Significantly **improves performance** of downstream NLP tasks such as classification, retrieval, and summarization
- Partially **addresses the sparsity problem** caused by Zipf's Law: most words in a corpus are rare

Key insight

- Preprocessing is not just technical - it is an analytical decision that shapes your results

Noise Removal

- Removing noise and irrelevant elements from raw text that are not relevant to the analysis
- Especially important for web-scraped data and social media content

Common cleaning operations:

- Remove HTML tags and special characters using regular expressions
- Handle URLs and email addresses (remove or replace with placeholders like [URL], [EMAIL])
- Standardize or remove numbers and dates (replace with tokens like [NUM], [DATE])
- Handle emojis and emoticons (remove or convert to text descriptions)
- Remove extra whitespace, tabs, and newline characters

Example:

- Input: " Check https://example.com on 15/04/2023! "
- Output: "Check [URL] on [NUM]/[NUM]/[NUM]"

Remove completely

Replace with placeholders

Preserve and standardize

Removing noice with Python and Regular Expressions (RegEx)

Removing HTML tags:

- re.sub('<.*?>', '', text) # Strips all HTML tags

Removing special characters:

- re.sub('[^a-zA-Z0-9\s]', '', text) # Keeps only alphanumeric + spaces

Replacing URLs:

- re.sub(r'http\S+|www\S+|https\S+', '[URL]', text)

Replacing email addresses:

- re.sub(r'\S+@\S+', '[EMAIL]', text)

Standardizing numbers:

- re.sub(r'\d+', '[NUM]', text)

Handling emojis (using emoji library):

- emoji.demojize(text) # Converts emojis to text descriptions

```python
def remove_urls_emails(text):
    # Remove URLs
    text = re.sub(r'http\S+|www\S+|https\S+','[URL]', text)
    # Remove emails
    text = re.sub(r'\S+@\S+', '[EMAIL]', text)
    return text
sample = "Visit https://example.com or email info@example.com"
result = remove_urls_emails(sample)
print(result)
# Output: Visit [URL] or email [EMAIL]
```

Text Normalization

- Normalization harmonizes different written forms of words with the same meaning

Common normalization operations:

- Deleting periods: U.S.A. → USA
- Deleting hyphens: anti-discriminatory → antidiscriminatory
- Handling accents: French résumé → resume
- Handling umlauts: German Tuebingen → Tübingen

Case folding:

- Converting all text to lowercase reduces vocabulary size
- Generally helpful, but can cause ambiguity in some cases
  - General Motors vs. general motors
  - CAT (City Airport Train) vs. cat (animal)
  - Fed (Federal Reserve) vs. fed (past tense of feed)

Text Normalization - Segmentation

- Segmentation splits compound words or contractions into individual tokens

French contractions:

  - L'ensemble → should it be one token or two? L? L'? Le?

German compound nouns:

  - Halsschlagader (carotid artery) → Hals + Schlag + Ader
- German requires specialized compound splitters using recursive binary splitting algorithms

Chinese and Japanese:

- No spaces between words - segmentation is even more critical
- Statistical and neural approaches are used for word boundary detection

Key consideration:

- Segmentation decisions directly affect downstream processing and are language-dependent

Spell Checking

Spell checking is the process of automatically identifies, highlights, and often corrects misspelled words in written text.

```
pip install pyspellchecker
```

Common Spell Checking Techniques in Text Cleaning

- ***Dictionary Lookup:*** Comparing words against a known list of correctly spelled words.
- ***Edit Distance (Levenshtein):*** Calculating the minimum number of insertions, deletions, or substitutions required to transform a misspelled word into a dictionary word.
- ***Context-Sensitive Correction:*** Using algorithms (like N-grams) to detect errors that are real words but contextually wrong, such as "Their coming too sea if its reel".
- ***Phonetic Algorithms:*** Identifying errors based on how words sound, which is helpful for fixing typos made while typing quickly.

```python
from spellchecker import SpellChecker
# 1. Start the spell checker
spell = SpellChecker()
# 2. Fix one misspelled word
fixed_word = spell.correction('recieve')
print(fixed_word)
# Output: receive
```

Tokenization

- Tokenization is the process of breaking text into smaller units called tokens
- Tokens are the fundamental unit of processing in NLP

Types of tokenization:

- Word tokenization: split on whitespace and punctuation
  - "The cat sat." → ["The", "cat", "sat", "."]
- Sentence tokenization: split text into individual sentences
  - Uses period, question mark, exclamation mark as delimiters
- Character-level tokenization: each character is a token
- Subword tokenization (BPE, WordPiece): used in modern transformers

Python example (NLTK):

- from nltk.tokenize import word_tokenize, sent_tokenize
- word_tokenize("This is a test.") → ['This', 'is', 'a', 'test', '.']

Tokenization - Language Hierarchy and Key Units

Linguistic perspective:

- Phonemes → Morphemes → Words → Phrases → Sentences
  - Morpheme: smallest meaningful unit (e.g., "national" = nation + -al)

Computational perspective:

- Character → N-gram character → Word → N-gram word → Sentence → Document → Corpus
  - Token: the chosen unit of processing (can be any of the above)
  - Vocabulary (Dictionary): list of unique tokens in the text
  - Corpus: a collection of text documents

N-gram examples:

  - Tri-gram characters of "fox": " fo", "fox", "ox "
  - Tri-gram words: "a fox jumped", "fox jumped over"

Tokenization - Language Hierarchy and Key Units

Installation: `pip install nltk`

Word Tokenization

```
from nltk.tokenize import word_tokenize
import nltk
nltk.download('punkt')
deftokenize_words(text):
    return word_tokenize(text)
sample = "This is a sample sentence for word tokenization."
tokens = tokenize_words(sample)
print(tokens)
# Output: ['This', 'is', 'a', 'sample', 'sentence',
#          'for', 'word', 'tokenization', '.']
```

Sentence Tokenization

```
from nltk.tokenize import sent_tokenize
def tokenize_sentences(text):
    return sent_tokenize(text)
sample = "First sentence. Here's another. What about this?"
sentences = tokenize_sentences(sample)
print(sentences)
# Output: ['First sentence.', "Here's another.",
#          'What about this?']
```

Choose tokenization strategy based on your downstream task. For traditional ML, NLTK's word_tokenize works well. For deep learning, consider subword tokenization (BPE, WordPiece).

Stop Word Removal

- Stop words are the most common words in a language: the, a, and, to, is, be, ...
- They carry little or no semantic information on their own

Why remove them?

- Reduces noise in bag-of-words approaches
- Decreases vocabulary size and processing time
- Focuses analysis on content-bearing words

But stop words can be important:

- In phrases: "King of Denmark", "To be or not to be"
- In titles: "Let it be"
- For definitional purposes: "flights to London" vs. "flights from London"

Python:

- from nltk.corpus import stopwords
- stop_words = set(stopwords.words('english'))

When to Remove Stop Words

✅ **Topic Modeling**
Focus on content words

✅ **Document Classification**
Reduce dimensionality

❌ **Sentiment Analysis**
"not good" vs "good"

❌ **Machine Translation**
Grammar structure needed

```python
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
import nltk
nltk.download('stopwords')

defremove_stopwords(text):
    stop_words = set(stopwords.words('english'))
    word_tokens = word_tokenize(text)

    filtered_text = [
        word for word in word_tokens
        if word.lower() notin stop_words
    ]

    return' '.join(filtered_text)
# Example
sample = "This is a sample sentence with some stop words."
result = remove_stopwords(sample)
print(result)
# Output: This sample sentence stop words .
```

Stemming

- A stemmer reduces words to their stem by crude affix chopping
- English morphemes consist of a stem (core meaning) and affixes (prefixes/suffixes)

Porter's Stemmer - most common algorithm for English:

- sses → ss (e.g., caresses → caress)
- ies → i (e.g., ponies → poni)
- ational → ate (e.g., relational → relate)
- tional → tion (e.g., conditional → condition)

Example transformation:

- automate, automates, automatic, automation → automat

Limitations:

- Can produce non-real words ("quickly" → "quickli")
- May increase ambiguity by conflating different meanings

```
from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize
# Initialize stemmer
stemmer = PorterStemmer()
defstem_words(text):
    words = word_tokenize(text)
    return' '.join([stemmer.stem(word)
                        for word in words])
sample = "The runner runs quickly through the running t
rack"
result = stem_words(sample)
print(result)
# Output: The runner run quickli through the run track
```

Lemmatization

- A lemmatizer uses a knowledge resource (like WordNet) to find correct base forms

Examples:

- am, are, is → be
- car, cars, car's, cars' → car
- children → child
- "the boy's cars are different colors"
- → "the boy car be different color"

```
from nltk.stem import WordNetLemmatizer
import nltk
nltk.download('wordnet')
# Initialize lemmatizer
lemmatizer = WordNetLemmatizer()
deflemmatize_words(text):
    words = word_tokenize(text)
    return' '.join([lemmatizer.lemmatize(word)
                        for word in words])
sample = "The children are playing with their toys"
result = lemmatize_words(sample)
print(result)
# Output: The child be playing with their toy
```

Key differences from stemming:

- Produces linguistically correct base forms (real words)
- Uses dictionary/knowledge base lookup rather than rules
- Typically slower than stemming but more precise
- If a word is not in the knowledge resource, lemmatization may fail to process it

Python:

- from nltk.stem import WordNetLemmatizer

Stemming vs. Lemmatization

Feature	Stemming	Lemmatization
Approach	Rule-based affix removal	Dictionary/knowledge-based lookup
Output	May produce non-words ("quickli")	Always produces real words ("quickly")
Speed	Faster (simple rules)	Slower (dictionary lookup)
Precision	Lower - may conflate meanings	Higher - context-aware
Example: "better"	better → better (no change)	better → good
Example: "running"	running → run	running → run
Use case	Information retrieval, search	Text analysis, NLU tasks

Text Representation Techniques

Bag-of-Words (BoW)

BoW represents text as word frequency counts, ignoring grammar and word order.

BoW Matrix Example:

	doc	first	is	second	the
Doc1	1	1	1	0	1
Doc2	2	0	1	1	1

Advantage: Simple and fast.

TF-IDF

TF-IDF weighs word importance by frequency in document vs. rarity across corpus.

TF-IDF Formula:

$$\text{TF-IDF} = \text{TF} \times \text{IDF}$$

TF = (Word count in doc) / (Total words in doc)
IDF = log(Total docs / Docs containing word)

Advantage: Reduces impact of common words, highlights distinctive terms

Word Embeddings

Dense vector representations capturing semantic relationships between words.

Popular Pre-trained Models:

- **Word2Vec** (2013)
- **GloVe** (2014)
- **FastText** (2016)
- **ELMo** (2018)
- **BERT** (2018)

Key Property: Capture sementic, similar words have similar vectors (e.g., king - man + woman ≈ queen)

Lab